

// Jivops

LLMOps en Producción

Guía completa — 30 lecciones

Fundamentos · Prompt engineering · RAG · Evaluación y calidad · Serving y optimización · Observabilidad y seguridad

Contenido

Módulo 1: Fundamentos de LLMOps

- Lección 01 — Qué es LLMOps y en qué se diferencia de MLOps clásico
- Lección 02 — Arquitectura de una aplicación LLM en producción
- Lección 03 — Elegir entre modelo propietario (API) vs modelo open-source autoalojado
- Lección 04 — Tokens, contexto y costos — unidades económicas de un LLM
- Lección 05 — Entornos: desarrollo, staging y producción para apps LLM

Módulo 2: Prompt engineering y gestión

- Lección 06 — Prompt engineering — principios básicos
- Lección 07 — Versionado de prompts como código
- Lección 08 — Plantillas de prompts y variables dinámicas
- Lección 09 — Few-shot prompting y ejemplos en el prompt
- Lección 10 — Prompt injection — riesgos y mitigaciones

Módulo 3: RAG

- Lección 11 — Qué es RAG y por qué se usa en producción
- Lección 12 — Embeddings y bases de datos vectoriales
- Lección 13 — Chunking — estrategias de división de documentos
- Lección 14 — Re-ranking y mejora de relevancia de recuperación
- Lección 15 — Actualización del índice — RAG con datos frescos

Módulo 4: Evaluación y calidad

- Lección 16 — Evaluación de LLMs — métricas y desafíos
- Lección 17 — LLM-as-judge — usar un modelo para evaluar a otro
- Lección 18 — Datasets de evaluación (golden sets)
- Lección 19 — Testing de regresión para prompts y modelos
- Lección 20 — A/B testing de prompts y modelos en producción

Módulo 5: Serving y optimización

- Lección 21 — Serving de LLMs — vLLM y motores de inferencia
- Lección 22 — Cuantización de modelos LLM
- Lección 23 — Streaming de respuestas — UX y arquitectura
- Lección 24 — Caching de respuestas y prompts
- Lección 25 — Fine-tuning vs prompting — cuándo usar cada uno

Módulo 6: Observabilidad, seguridad y avanzado

Lección 26 — Observabilidad de aplicaciones LLM — tracing de llamadas

Lección 27 — Guardrails — filtrado de contenido y validación de salidas

Lección 28 — Gestión de costos a escala — rate limiting y budgets

Lección 29 — Agentes LLM en producción — orquestación de herramientas

Lección 30 — Proyecto final: plataforma LLMOps completa lista para producción

Módulo 1: Fundamentos de LLMOps

Lección 01 — Qué es LLMOps y en qué se diferencia de MLOps clásico

LLMOps añade preocupaciones específicas como gestión de prompts, costos por token, guardrails de contenido y evaluación cualitativa de texto generado, en vez de solo métricas numéricas como accuracy.

Un modelo de clasificación produce una etiqueta fija fácil de comparar contra la verdad; un LLM genera texto libre donde puede haber múltiples respuestas igualmente válidas, dificultando una métrica automática simple.

Lección 02 — Arquitectura de una aplicación LLM en producción

Una capa de orquestación de prompts, posiblemente un sistema de recuperación de datos (RAG), guardrails de validación, y observabilidad de cada llamada.

Si la lógica de armar el prompt y procesar la respuesta vive separada de la llamada específica al modelo, cambiar de un proveedor a otro (o de un modelo a otro) requiere modificar solo esa integración puntual, sin reescribir toda la lógica de negocio de la aplicación.

Lección 03 — Elegir entre modelo propietario (API) vs modelo open-source autoalojado

Mayor control sobre los datos (que nunca salen de la infraestructura propia) y sobre el costo a largo plazo en escenarios de alto volumen de uso.

Mantener los datos sensibles completamente dentro de su propia infraestructura, sin enviarlos nunca a un servicio externo de terceros, puede ser un requisito de cumplimiento regulatorio ineludible en sectores como salud o finanzas, justificando el costo adicional de autoalojar.

Lección 04 — Tokens, contexto y costos — unidades económicas de un LLM

Es la cantidad máxima de tokens (entrada más salida) que el modelo puede procesar en una sola llamada; la mayoría de proveedores cobran proporcionalmente a la cantidad de tokens usados.

Si cada nueva llamada incluye todo el historial previo de la conversación (necesario para que el modelo tenga contexto), la cantidad de tokens procesados crece con cada turno, aumentando el costo de cada llamada sucesiva a medida que la conversación se alarga.

Lección 05 — Entornos: desarrollo, staging y producción para apps LLM

Usar modelos y datasets de evaluación consistentes entre entornos, para que un cambio de prompt validado en staging se comporte de forma predecible al pasar a producción.

Las pruebas manuales en desarrollo suelen cubrir solo unos pocos casos anticipados por el desarrollador, mientras la variedad real de peticiones de usuarios en producción puede exponer patrones de entrada completamente distintos que el prompt no maneja bien.

Módulo 2: Prompt engineering y gestión

Lección 06 — Prompt engineering — principios básicos

Reduce la ambigüedad sobre qué se espera exactamente, disminuyendo la variabilidad de interpretación que el modelo podría hacer de una instrucción poco clara.

Una respuesta en un formato estructurado y predecible (como JSON) puede parsearse programáticamente de forma directa y confiable, mientras una respuesta de texto libre requeriría lógica adicional para extraer la información relevante de forma consistente.

Lección 07 — Versionado de prompts como código

Un cambio de prompt puede alterar significativamente el comportamiento de la aplicación, así que necesita el mismo rigor de revisión, historial y posibilidad de rollback que un cambio de código.

En vez de tener que depurar en vivo qué salió mal con el prompt nuevo mientras los usuarios siguen recibiendo respuestas de peor calidad, se puede revertir de inmediato a la versión anterior ya conocida y confiable, minimizando el impacto negativo mientras se investiga con calma.

Lección 08 — Plantillas de prompts y variables dinámicas

Centraliza la estructura del prompt en un solo lugar legible, facilitando su revisión, versionado y modificación sin tener que rastrear código de construcción de strings disperso.

Al tener el texto del prompt centralizado y legible en un solo lugar (en vez de fragmentado dentro de lógica de concatenación de strings en el código), alguien sin conocimientos de programación puede leerlo y sugerir mejoras directamente sobre ese texto.

Lección 09 — Few-shot prompting y ejemplos en el prompt

Guiar al modelo hacia el formato y estilo de respuesta deseado mostrándole ejemplos concretos, sin necesitar reentrenar o ajustar los pesos del modelo.

Un ejemplo concreto demuestra exactamente el patrón esperado de forma directa, mientras una descripción puramente verbal del formato deja más margen de interpretación ambigua sobre los detalles exactos que se esperan en la respuesta.

Lección 10 — Prompt injection — riesgos y mitigaciones

Un usuario malicioso incluye instrucciones dentro de su entrada que intentan hacer que el modelo ignore las instrucciones originales del sistema y siga en cambio las instrucciones inyectadas por el atacante.

Si el contenido del documento se incorpora directamente al prompt sin ningún tratamiento especial, el modelo podría no distinguir entre el texto legítimo a resumir y una instrucción maliciosa incrustada deliberadamente dentro de ese mismo documento por un atacante.

Módulo 3: RAG

Lección 11 — Qué es RAG y por qué se usa en producción

Permite que el modelo genere respuestas basadas en información específica y actualizada (como documentos internos de una empresa) que no formaba parte de sus datos de entrenamiento originales, sin necesitar reentrenar el modelo.

Reentrenar un modelo completo es costoso en tiempo y recursos computacionales; con RAG, basta con actualizar el índice de documentos recuperables, sin tocar el modelo en sí, haciendo mucho más ágil mantener la información actualizada.

Lección 12 — Embeddings y bases de datos vectoriales

Una representación numérica (vector) del significado semántico del texto, de forma que textos con significado similar tengan vectores cercanos entre sí en ese espacio numérico.

Dos textos pueden expresar la misma idea con palabras completamente distintas (sinónimos, paráfrasis); una búsqueda por coincidencia exacta de palabras clave no los relacionaría, mientras la similitud de embeddings captura esa cercanía de significado aunque las palabras literales sean diferentes.

Lección 13 — Chunking — estrategias de división de documentos

Un embedding de un documento entero muy largo diluiría el significado específico de cada parte, y además la ventana de contexto del LLM tiene un límite que no podría absorber el documento completo de una vez.

Una frase aislada de su párrafo original podría perder referencias importantes (como a qué se refiere un pronombre, o qué tema se estaba discutiendo), haciendo que ese fragmento recuperado sea ambiguo o incompleto sin el contexto circundante que tenía originalmente.

Lección 14 — Re-ranking y mejora de relevancia de recuperación

Reordena los resultados iniciales usando un modelo más preciso (aunque más costoso computacionalmente) para mejorar la relevancia final antes de enviar los mejores fragmentos al LLM.

La búsqueda vectorial inicial (más barata computacionalmente) ya filtra el corpus completo hacia un subconjunto reducido de candidatos plausibles; aplicar el modelo de re-ranking más costoso solo a ese subconjunto pequeño, en vez de a todo el corpus, mantiene el costo total razonable mientras mejora la precisión final.

Lección 15 — Actualización del índice — RAG con datos frescos

Un pipeline automatizado que detecte cambios en los documentos fuente y regenere/actualice solo los embeddings de los fragmentos afectados, en vez de reconstruir todo el índice desde cero cada vez.

Regenerar los embeddings de miles de documentos que no cambiaron, solo porque uno específico se actualizó, desperdiciaría recursos computacionales significativos de forma innecesaria; una actualización incremental que solo toca los fragmentos realmente afectados es mucho más eficiente.

Módulo 4: Evaluación y calidad

Lección 16 — Evaluación de LLMs — métricas y desafíos

Penalizan respuestas que son correctas y útiles pero están redactadas de forma distinta a la referencia exacta, sin capturar bien la equivalencia semántica o la calidad subjetiva de la respuesta.

BLEU mide principalmente la superposición literal de palabras/frases entre el texto generado y la referencia; dos respuestas con el mismo significado correcto pero expresadas con palabras y estructura muy distintas tendrían poca superposición textual, resultando en una puntuación baja a pesar de ser igualmente válidas.

Lección 17 — LLM-as-judge — usar un modelo para evaluar a otro

Usar un LLM (normalmente uno más potente o especializado) para juzgar automáticamente la calidad de las respuestas generadas por otro modelo, según criterios definidos en un prompt de evaluación, escalando mucho más que la revisión humana manual.

La revisión humana manual de miles de respuestas diarias sería extremadamente costosa en tiempo y recursos; un LLM-as-judge puede evaluar ese mismo volumen de forma automatizada y consistente, dando cobertura mucho más amplia de lo que sería posible con revisión humana exclusivamente.

Lección 18 — Datasets de evaluación (golden sets)

Un conjunto curado de preguntas o casos de prueba con respuestas esperadas de alta calidad, usado como referencia consistente para evaluar el desempeño del sistema a lo largo del tiempo.

Si el conjunto de casos de prueba cambiara constantemente, sería imposible saber si una diferencia en la puntuación se debe a un cambio real en la calidad del sistema o simplemente a que se están evaluando casos distintos; un golden set estable permite una comparación justa y consistente entre versiones.

Lección 19 — Testing de regresión para prompts y modelos

Que el cambio no haya degradado el desempeño en casos que antes funcionaban correctamente, comparando los resultados nuevos contra un baseline conocido del golden set.

Aunque el prompt no cambie, una nueva versión del modelo del proveedor podría interpretar y responder a ese mismo prompt de forma distinta a la versión anterior, potencialmente introduciendo una regresión de calidad que solo el testing automatizado detectaría a tiempo.

Lección 20 — A/B testing de prompts y modelos en producción

Medir el impacto real de cada versión en métricas de negocio relevantes (como satisfacción del usuario o tasa de conversión), no solo en un benchmark aislado, antes de decidir cuál adoptar completamente.

El golden set, aunque útil, es una muestra curada y limitada que no necesariamente captura toda la variedad y el comportamiento real de los usuarios finales; el A/B testing en producción mide el impacto genuino sobre métricas de negocio con usuarios reales, algo que una evaluación offline no puede replicar completamente.

Módulo 5: Serving y optimización

Lección 21 — Serving de LLMs — vLLM y motores de inferencia

Técnicas como PagedAttention que gestionan de forma más eficiente la memoria usada durante la generación de texto, permitiendo atender más peticiones concurrentes con el mismo hardware.

Si el mismo hardware costoso de GPU puede atender un número mayor de peticiones simultáneas gracias a un uso más eficiente de la memoria, el costo fijo de esa GPU se distribuye entre más peticiones, reduciendo el costo promedio por cada respuesta individual generada.

Lección 22 — Cuantización de modelos LLM

Se reduce el uso de memoria y se acelera la inferencia, a cambio de una posible pequeña pérdida de calidad en las respuestas generadas, que suele ser aceptable en muchos casos de uso.

En un caso de uso donde una pequeña imprecisión ocasional tiene consecuencias mínimas, el ahorro de costo y velocidad de la cuantización compensa claramente ese riesgo menor; en un caso crítico donde un error podría tener consecuencias graves, ese mismo tradeoff podría no ser aceptable.

Lección 23 — Streaming de respuestas — UX y arquitectura

El usuario empieza a ver texto apareciendo casi de inmediato, reduciendo la percepción de espera aunque el tiempo total de generación sea el mismo.

Ver contenido apareciendo de inmediato confirma al usuario que el sistema está funcionando activamente y da una sensación de progreso continuo, mientras una pantalla en blanco sin ninguna señal genera incertidumbre sobre si la aplicación está realmente procesando algo, aunque el tiempo real de espera total sea idéntico.

Lección 24 — Caching de respuestas y prompts

Peticiones con el mismo prompt exacto (o muy similar) que se repiten con frecuencia, donde la respuesta esperada no cambia significativamente entre una llamada y otra.

Una pregunta que se repite frecuentemente y de forma idéntica entre muchos usuarios distintos se beneficia enormemente de servir la misma respuesta cacheada una y otra vez, mientras una pregunta única y personalizada por cada usuario nunca tendría una coincidencia previa en la caché que aprovechar.

Lección 25 — Fine-tuning vs prompting — cuándo usar cada uno

Cuando se necesita que el modelo internalice un estilo, formato o conocimiento de dominio muy específico de forma consistente en un volumen muy alto de casos, algo que el prompting por sí solo no logra de forma suficientemente confiable.

Ajustar un prompt es una iteración rápida y de bajo costo que puede probarse en minutos, mientras el fine-tuning requiere preparar datos de entrenamiento, tiempo de cómputo y validación adicional; tiene sentido agotar primero la opción más económica antes de invertir en la más costosa.

Módulo 6: Observabilidad, seguridad y avanzado

Lección 26 — Observabilidad de aplicaciones LLM — tracing de llamadas

El flujo completo de la petición a través de cada componente (recuperación, construcción del prompt, llamada al modelo, post-procesamiento), con su latencia y resultado en cada etapa.

Sin ver los pasos intermedios, solo se sabría que la respuesta final fue mala, sin poder distinguir si el problema vino de recuperar documentos poco relevantes o de que el modelo generó una mala respuesta incluso con buen contexto disponible; el tracing detallado permite aislar exactamente en qué etapa específica ocurrió el problema.

Lección 27 — Guardrails — filtrado de contenido y validación de salidas

Valida que la respuesta generada cumpla ciertas reglas (formato esperado, ausencia de contenido dañino, ausencia de información sensible filtrada) antes de dejarla pasar hacia el usuario.

Si una respuesta con formato inválido pasara sin ninguna validación hacia el siguiente componente del sistema (que espera un formato específico), ese componente probablemente fallaría al intentar procesarla; el guardrail detecta el problema en el origen, antes de que se propague en cascada hacia el resto del sistema.

Lección 28 — Gestión de costos a escala — rate limiting y budgets

Evita que un usuario individual (por error, abuso, o un bug en su propio código integrador) genere un volumen desproporcionado de llamadas costosas al modelo, protegiendo tanto el presupuesto como la disponibilidad para otros usuarios.

Sin ningún límite que frene ese comportamiento erróneo, el loop infinito seguiría generando llamadas reales y costosas al modelo de forma indefinida, acumulando un costo que podría crecer muy rápidamente antes de que alguien note el problema manualmente.

Lección 29 — Agentes LLM en producción — orquestación de herramientas

Puede decidir de forma autónoma qué herramientas externas invocar (como buscar en internet, consultar una base de datos, ejecutar código) para completar una tarea compleja de múltiples pasos.

Mientras una aplicación LLM simple solo genera texto que un humano luego interpreta, un agente con acceso a herramientas puede ejecutar acciones reales de forma autónoma; si el modelo razona incorrectamente sobre cuándo o cómo usar una herramienta destructiva, esa decisión errónea se traduciría en un daño real al sistema, no solo en una respuesta de texto equivocada sin consecuencias directas.

Lección 30 — Proyecto final: plataforma LLMOps completa lista para producción

Prompts versionados y probados + RAG con índice actualizado + evaluación continua con golden set y LLM-as-judge + serving optimizado + observabilidad con tracing + guardrails y control de costos.

Intenta mencionar al menos dos pilares: prompts versionados y probados, evaluación continua con golden set, RAG actualizado o guardrails con control de costos.